

CS4225/5425 Big Data Systems, Data Science

AY24/25 Sem 2. github.com/gerteck
(Big) Data science. Interdisciplinary field about processes and systems to extract knowledge or insights from data in various forms.

- **Processing** diff. types of data (large data volume, graph data, stream data).
- **Models of Computation:** MapReduce/Spark, Large graph processing engines, Streams and online algorithms.

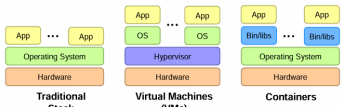
1. Introduction

- **Big Data (4 ‘V’s):** *Volume:* (Scale of data). *Velocity:* (Speed of new (streaming) data). *Variety:* (Different types). *Veracity:* (Accuracy, trustworthiness of data).
- **Challenges of 4 ‘V’s:** Performance, Cost, Reliability, Algorithm design complexity. Data integration challenges, multi-modal learning (Variety). Dirty and noisy data, data provenance, uncertainty (Veracity).
- Big data - Virtuous Product Cycle, where useful service → data to analyze → insightful action → useful service.

Utility Computing (Cloud Computing Infrastructure)

Cloud computing - scalable, elastic infrastructure for big data systems.

- **Utility Computing:** Computing resources as a metered service (“pay as you go”). Ability to dynamically provision virtual machines.
- **Scalable:** (“Infinite” capacity). **Elastic:** (Scale up or down on demand).



- **How:** **Virtual Machines (VMs):** Enable sharing of hardware resources, run each app in isolated virtual machine. High overhead as each VM has its own OS. **Containers:** Lightweight sharing of resources. Apps run in isolation but share same OS. Container encapsulates an app and environment.
- **Everything as a Service:**
 - *Infrastructure (IaaS):* User rents a virtual machine. (e.g. Amazon EC2).
 - *Platform (PaaS):* Provide hosting web apps. Cover hardware maintain, upgrade.
 - *Software (SaaS):* Pre-built apps. (e.g., Gmail). Covers entire stack.

Data science, cross-disciplinary, emerging research area w. applications in engineering, commerce, etc. Clouds as natural infrastructures for data science.

2. Principles of Big Data Systems

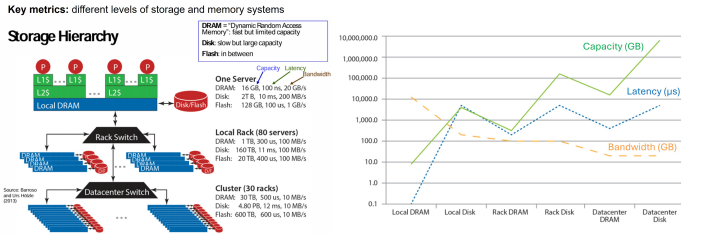
- **Data Center Infrastructure**
 - **Single Server:** A commodity hardware unit.
 - **Rack:** Dozens of servers, connect to rack switch (top-of-rack switch).
 - **Data Center:** Hundreds of racks, connect to data center switch (core switch).
- **Bandwidth vs. Latency**
 - **Bandwidth:** Max amt of data transmittable per unit time (e.g., in GB/s). Calculate as minimum bandwidth among all components in the data access path.
 - **Latency:** Time taken one packet travel src to dest (one-way) / round-trip (in ms). Sum of the latency caused by all components in the data access path.
 - **Throughput:** Rate data is actually transmitted in network in given time period.
- When transmitting (*large/small*) amount of data, (*bandwidth/latency*) - roughly how long transmission will take (bottleneck variable).
- **Storage Hierarchy in Data Centers:** based on capacity, latency, bandwidth.
 - **DRAM:** Fast but limited capacity. **Flash:** Intermediate btw. DRAM and disk.
 - **Disk:** Slow but large capacity. Disk reads more expensive than DRAM (higher latency, lower bandwidth).

Data Flow Paths

Data flow paths depend on the location of the source and destination. Data access may cross several components, e.g. hard disk, DRAM, rack switch, cluster switch.

- E.g. Local DRAM → Local Disk in same server.
- Server A DRAM to server B DRAM in same rack (i.e. Local DRAM → Rack Switch → Server B’s DRAM)
- Transfer across different rack, same cluster etc. (i.e. Local DRAM → Rack Switch → Datacenter Switch → Server C’s Rack Switch → Server C’s DRAM).
- To load from disk, usually need to load into DRAM first. (i.e. Local Disk → Local DRAM → Rack Switch etc.)

Calculate latency, bandwidth based on data flow path. Approximate order of magnitude, for simplify, assume one-way communication, no failures.



Capacity increases, Costs (Latency increase, bandwidth decrease) increases from local to rack to data center, along storage hierarchy.

Building Efficient Big Data Systems

1. **Scale “Out”, Not “Up”:** Horizontal Scaling over vertical scaling. Combine many cheaper machines (out) over increase power of individual machine (up).
2. **Seamless Scalability:** Ideal (linear) scalability, performance increases proportionally to added resources.
3. **Move Processing to the Data:** As clusters limited bandwidth; move tasks to machine where data is stored.
4. **Process Data Sequentially, Avoid Random Access:** Disk seeks expensive; sequential access provides better throughput.

Challenges in Big Data Systems

1. **Machine Failures:** With enough scale, expect daily server machine failure.
2. **Synchronization:** Difficult due to unknown order of worker execution, interruptions, and shared data access. Tools like semaphores, conditional variables, and barriers are used to address synchronization issues.
3. **Programming Difficulty:** Concurrency hard to reason about, especially at scale. Debugging and managing failures add complexity.

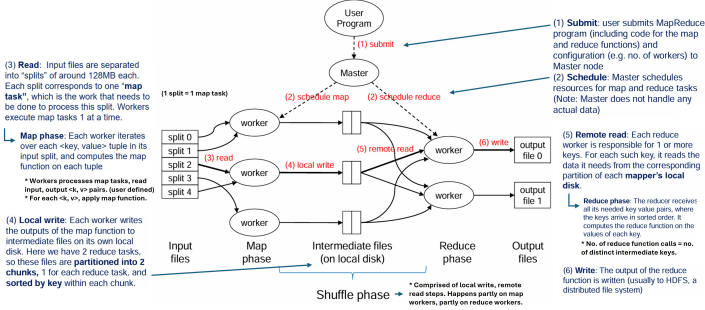
API for Data Centers: Given challenges, “API” of data center abstracts system-level details from developers. Execution frameworks (e.g., Hadoop, Spark) handle task assignment, replication, synchronization, etc.

3. MapReduce

MapReduce: prog. model, associated runtime system to process large datasets in parallel across distributed cluster. (Parallelization, Divide Conquer Paradigm). Provide functional abstractions for two operations.

- **Map function:** (k1, v1) → List(k2, v2). Processes each input key-value pair and emits intermediate key-value pairs.
- **Reduce:** Aggregate intermediate results to generate the final output. reduce(k2, List(v2)) → List(k3, v3). All values with same key sent to same reducer.

- **Typical Workflow of MapReduce:** Iterate over a large number of records. Extract relevant information from each (**Map**). Shuffle and sort intermediate results by key (**Shuffle**). Aggregate intermediate results (**Reduce**). Generate the final output.



- **Worker:** single component of cluster, performs storage, processing tasks.
- **Map Task:** A unit of work, processing one input split (e.g., 128 MB).
- **Map Function:** A call to user-defined map function, for each input split k-v pair.
- **Reduce Task:** A unit of work processing a subset of intermediate keys.
- **Barrier:** barrier between map and reduce phases. Else, reduce phase may compute wrong answer. Shuffle phase can begin copying intermediate data earlier.

Partition and Combiner

Additionally, can optionally specify **partition** and **combine** functions as optimization to reduce network traffic.

- **Partition Step:** By default, keys assigned to reducers using hash function (e.g., hash(key) % num_reducers). Can implement custom partitioner to balance load among reducers. (E.g. if some keys much more values than others).
- Partitioner runs in local write phase, knowledge of which keys to which reducer.
- **Combiner Step:** Combiners locally aggregate map outputs to reduce disk and network I/O. (‘mini-reducers’) (E.g. Sum counts for each key before write to disk). *Correctness:* Must not affect correctness of final output, regardless of how many times it runs (incl. 0 times). E.g. sum, max ok, but mean, minus not ok.
- Combiner runs after map phase, done during local write phase, before actual disk write to save disk I/O.

Notes

- If reduce task handles multiple keys, will process keys in sorted order.
- If size of split too big or small: Limited parallelism vs. high overhead.
- #map fn calls = # input key-value pairs. # map tasks = # input splits.
- Reduce function calls = no. of distinct intermediate keys.
- **Preserving State in Map / Reduce Tasks:** In MapReduce, state variables can be used to store information that persists across multiple calls to map() or reduce() functions within same task. Sharing data or resources between function calls.

General Performance Guidelines (Basic Algorithm Design)

- **Linear Scalability:** More nodes should enable more work done in same time.
- **Minimize Disk and Network I/O:** Sequential access over random access. Send data in bulk over small chunks.
- **Reduce Memory Working Set:** Avoid large in-memory data structures to prevent out-of-memory errors.

Big data requires new programming abstractions and runtime systems (e.g., MapReduce). Hadoop MapReduce is widely used for efficiently processing large datasets in applications like databases and data mining.